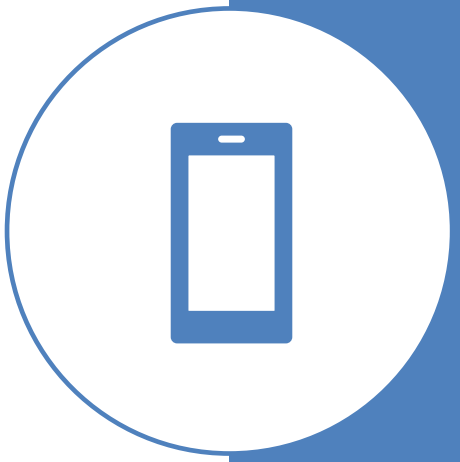
Mobile App and Python script

Dr. Salwa Osama

Mobile APP

- A mobile application is a software program that runs on smartphones.
  There are two main types:
  - **Android Applications** (used on Samsung, Realme, Xiaomi, etc.)
  - **iOS Applications** (used on iPhone devices)
- We will focus on **Android applications** because their development environment is free and easy to work with.
- **Tools Needed**
  - To start developing an Android application, we need:
    - **Android Studio** (Development IDE)
    - **Java Development Kit (JDK)**
    - **Android device or emulator** to test the application



What is Android Studio?

Android Studio is the official **Integrated Development Environment (IDE)** for **Android application development.**

It is a robust tool for developing high-quality **Android applications**. It includes all of the necessary tools for developing **Android apps** and also for **code development, testing,** and **deployment**.

Android Studio provides all of the tools needed for developers to create an **Android app**.

It is a **popular IDE** for producing high-quality **Android applications**, suitable for both **beginners and advanced developers.**

Android Studio Koala

- Develop a modern and professional application
- Easily integrate Python into the app later
- Avoid compatibility issues that appear in older versions

Create a New Project

Open Android Studio → **File → New → New Project**.

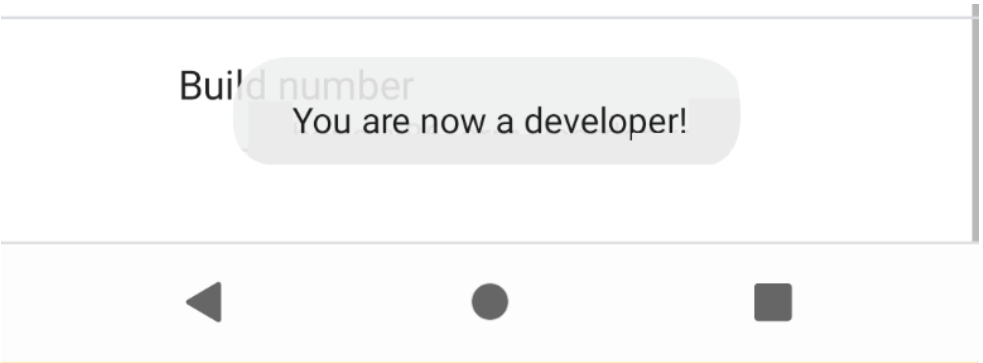
Choose **Empty Activity**.

Enter:

- Project Name (e.g., MyFirstApp)
- Package Name (e.g., com.example.myfirstapp)
- Save location
- Language: **Kotlin**
- Minimum SDK: 24+ (allows more devices to run your app)

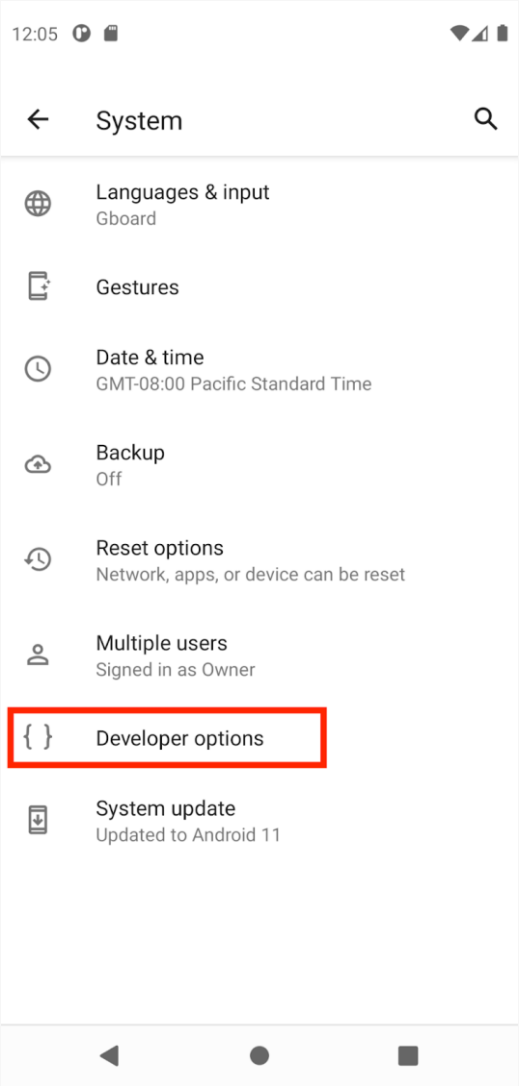
Connect Your Mobile Device

- To let Android Studio communicate with your Android device, you must enable USB debugging in the Developer options settings of the device.
- To show developer options and enable USB debugging:
  1. On your Android device, tap **Settings** > **About phone**.
  2. Tap **Build number** seven times.
  3. If prompted, enter your device password or pin. You know you succeeded when you see a **You are now a developer!** message.



Connect Your Mobile Device

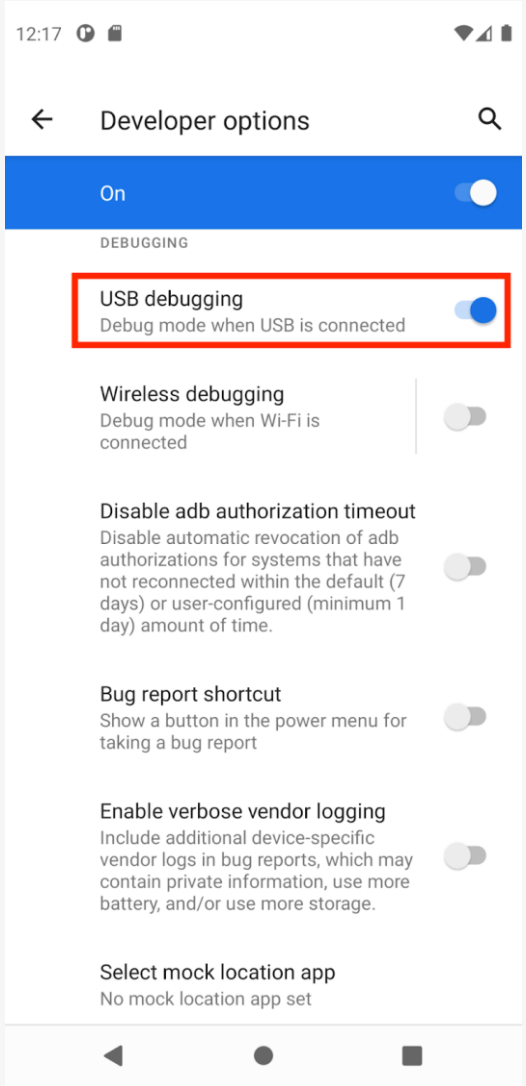
1. On your Android device, tap **Settings** > **About phone**.
2. Tap **Build number** seven times.
3. If prompted, enter your device password or pin. You know you succeeded when you see a **You are now a developer!** message.
4. Return to **Settings** and then tap **System** > **Developer options**.
5. If you don't see **Developer options**, tap **Advanced options**.



Connect Your Mobile Device

4. Return to **Settings** and then tap **System** > **Developer options**.
5. If you don't see **Developer options**, tap **Advanced options**.
6. Tap **Developer options** and then tap the **USB debugging** toggle to turn it on.
7. Or Using Allow Wireless Debugging

   Click Pair by QR in Visual Studio and wireless debugging -> pair by QR



Run Your Project on the Phone

- In Android Studio, click the **Run** button (green triangle).
- Select your connected device in the target device list.
- Wait for the project to compile and install on your phone.
- The app will launch automatically on your device.

How I can Add python

Open Your Existing Android Project

Make sure it compiles and runs on your device or emulator.

Add Chaquopy PluginIn build.

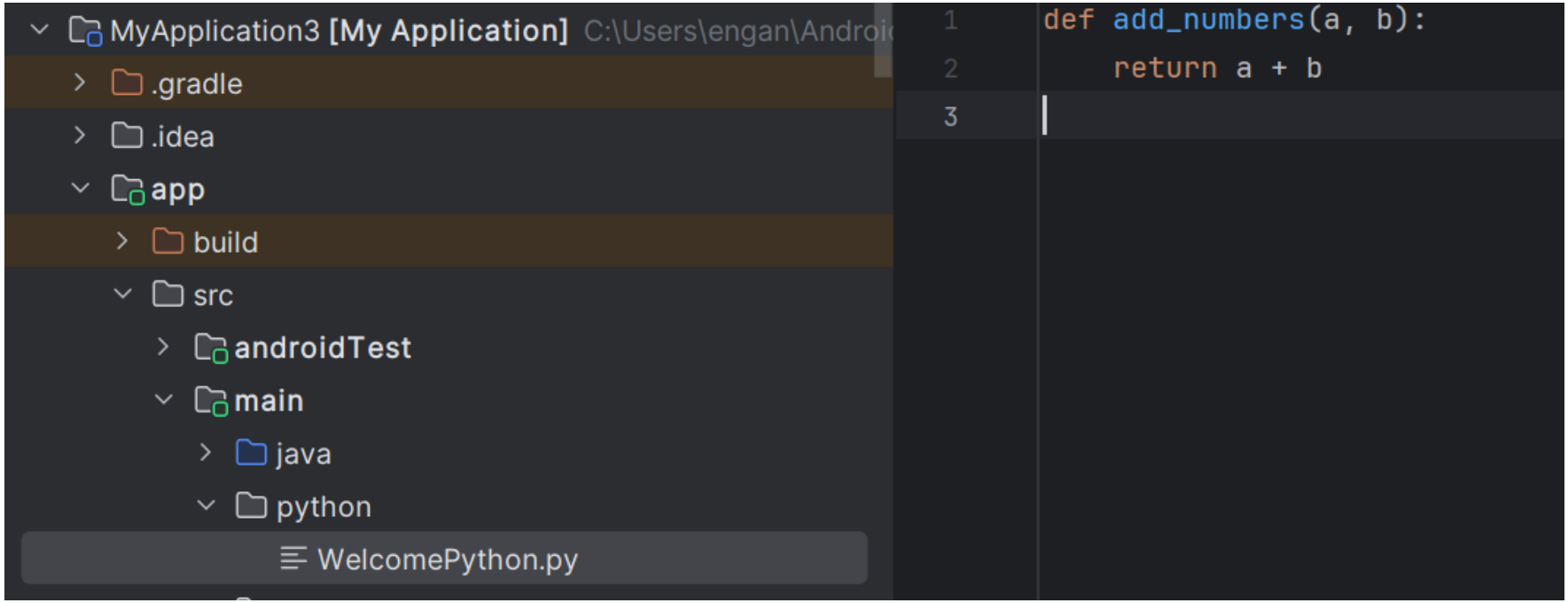
gradle.kts (Module: app):

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.jetbrains.kotlin.android)
    id("com.chaquo.python") version "16.0.0"
}
```

How I can Add python

Create Python Folder

Project View



How I can Add python

- **Call Python from Kotlin**
- **MainActivity.kt**

```kotlin
import com.chaquo.python.Python
import com.chaquo.python.android.AndroidPlatform
```

```kotlin
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        if (!Python.isStarted()) {
            Python.start(AndroidPlatform( context: this))
        }
        setContent {

            var resultText by remember { mutableStateOf( value: "please enter the button") }

            Column(modifier = Modifier.fillMaxSize().padding(24.dp)) {

                Text(text = resultText, modifier = Modifier.padding(bottom = 16.dp))

                Button(onClick = {
                    val py = Python.getInstance()
                    val module = py.getModule( name: "WelcomePython")
                    val result = module.callAttr( key: "add_numbers",  ...args: 5, 3)
                    resultText = "Result is: ${result.toString()}"
                }) {
                    Text( text: "Click Here")
                }
            }
```

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        if (!Python.isStarted()) {
            Python.startAndroidPlatform( context: this))
        }
        S
                                    remember { mutable
                                    Modifier.fillMaxS
                                    = resultText, modifier
            Button(onClick = {
                val py = Python.getInstance()
                val module = py.getModule( name: "WelcomePython")
                val result = module.callAttr( key: "add_numbers",  ...args: 5, 3)
                resultText = "Result is: ${result.toString()}"
            }) {
                Text( text: "Click Here")
            }
        }
```

This class **inherits** from ComponentActivity, which is a standard Android activity

This is the entry point of the activity. onCreate is called when the activity is first created.
savedInstanceState = A special box that can hold data about the screen if the app was **closed or rotated** before.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        if (!Python.isStarted()) {
            Python.start(AndroidPlatform( context: this))
        }
        setContent {

            var resultText by remember { mutableStateOf( value: "please enter the button") }

            Column(modifier = Modifier.fillMaxSize(

                Text(text = resultText, modifier =

                Button(onClick = {
                    val py = Python.getInstance()
                    val module = py.getModule( name
                    val result = module.callAttr( k
                    resultText = "Result is: ${result.toString()}"
                }) {
                    Text( text: "Click Here")
                }
            }
```

Calls the **parent class implementation** of onCreate.

Checks if the Python interpreter **has not started** yet.
If not started, it **starts Chaquopy's Python interpreter** for Android.
AndroidPlatform(this) tells Python to run in the context of this Android activity.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        if (!Python.isStarted()) {
            Python.start(AndroidPlatform(
        }
        setContent {

            var resultText by remember { mutableStateOf( value: "please enter the button") }

            Column(modifier = Modifier.fillMaxSize().padding(24.dp)) {

                Text(text = resultText, modifier =

                Button(onClick = {
                    val py = Python.getInstance()
                    val module = py.getModule( name
                    val result = module.callAttr( k
                    resultText = "Result is: ${resu
                }) {
                    Text( text: "Click Here")
                }
            }
```

This begins the **Jetpack Compose UI block**.
Everything inside this block defines what the UI will look like.

mutableStateOf("...") creates a variable whose changes are tracked.
Here, it starts with "please enter the button".
Later, if we assign something new to resultText, the UI will recompose (refresh) automatically.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        if (!Python.isStarted()) {
            Python.start(AndroidPlatform( context: this))
        }
        setContent {

            var resultText by remember { mutableStateOf( value: "please enter the button") }

                                                ize(

                                                r =

                                                ()
                                                name
                                                r( k
                resultText = "Result is: ${result.toString()}"
            }) {
                Text( text: "Click Here")
            }
        }
```

- resultText is a text variable shown on the screen.
- It **starts** as "please enter the button".
- If you change it in code (like after clicking the button), the UI **automatically updates** to show the new value.

In Compose, the UI can **rebuild many times**.
Without remember, the variable would **reset to its initial value every time the UI redraws**.
remember { ... } **keeps the value even when the UI is redrawn**.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundl
        super.onCreate(savedInstanceState)
        if (!Python.isStarted()) {
            Python.start(AndroidPlatform( context: th
        }
        setContent {

            var resultText by remember { mutableSta

            Column(modifier = Modifier.fillMaxSize().padding(24.dp)) {

                Text(text = resultText, modifier = Modifier.padding(bottom = 16.dp))

                Button(onClick = {
                    val py = Python.getInstance()
                    val module = py.getModule( na
                    val result = module.callAttr(
                    resultText = "Result is: ${re
                }) {
                    Text( text: "Click Here")
                }
            }
```

Creates a **vertical layout container** (Column) to place items **one below another**.
fillMaxSize() makes the column fill the entire screen.
padding(24.dp) adds 24 density-independent pixels of padding around the column.

Displays the **current value of resultText** on the screen.
Adds a **bottom padding** of 16.dp to separate it from the next element.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundl
        super.onCreate(savedInstanceState)
        if (!Python.isStarted()) {
            Python.start(AndroidPlatform( context: th
        }
        setContent {

            var resultText by remember { mutableSta

            Column(modifier = Modifier.fillMaxSize(

                Text(text = resultText, modifier =

                Button(onClick = {
                    val py = Python.getInstance()
                    val module = py.getModule( name: "WelcomePython")
                    val result = module.callAttr( key: "add_numbers",  ...args: 5, 3)
                    resultText = "Result is: ${result.toString()}"
                }) {
                    Text( text: "Click Here")
                }
            }
```

Gets the **instance of the Python interpreter** that we started earlier. This allows us to call Python code from Kotlin.

Loads a Python file named WelcomePython.py located in the src/main/python folder.
You can now call functions defined in this Python module.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundl
        super.onCreate(savedInstanceState)
        if (!Python.isStarted()) {
            Python.start(AndroidPlatform( context: th
        }
        setContent {

            var resultText by remember { mutableSt

            Column(modifier = Modifier.fillMaxSize(

                Text(text = resultText, modifier =

                Button(onClick = {
                    val py = Python.getInstance()
                    val module = py.getModule( name: "WelcomePython")
                    val result = module.callAttr( key: "add_numbers", ...args: 5, 3)
                    resultText = "Result is: ${result.toStri
                }) {
                    Text( text: "Click Here")
                }
            }
```

Calls the Python function add_numbers **with arguments 5 and 3**.
Stores the returned result in the result variable.

Updates the resultText variable to display the result.
Because resultText is a **state variable**, the UI **automatically updates** with this new value.

Adds a **label** inside the button saying "Click Here".

Painting Project but With mobile APP

1

Step 1: Install Pydroid 3

- Go to Google Play Store.
- Download Pydroid 3 – IDE for Python 3.

2

Step 2: Install Kivy inside Pydroid 3

- Open Pydroid 3.
- Go to PIP menu (package installer).
- Search for kivy and install it.
- Command in Pydroid terminal:
- pip install kivy

3

Step 3: Write your first touch screen demo

Painting Project but With mobile APP

File Edit Selection View ··· AI COdes

EXPLORER

- AI CODES
  - AI COdes
    - CommonStepsML
      - nox_setup_v7.0.2.1_full_intl.exe
      - Preprocessing.py
      - SignalPreprocessing.py
      - TextPreprocessing.py
      - WithIllumination.png
    - DataScience
    - DeepLearning
    - FunctionalProgramming
    - GUI
    - HCI Course
      - Accelerometer
        - AccelerationMobile.py
        - mouseAccelerometer.py
      - Basics of HCI
      - EyeTracking
      - Gaze_Controlled_Keyboard-main
      - TouchScreen
        - PaintExample.py
        - SimpleExample.py
      - VirtualKeyboard
      - TouchScreen.rar
      - VirtualKeyboard.rar
    - HelpfulCode
    - LinearAlgebraDS
    - MyResearch
      - KNN.py 3
      - LinearRegression.py
    - NLP CODE

city.py | k-mean.py | VideoCapture.py | FaceDetection.py | Eyedetection.py | pupilTra

AI COdes > HCI Course > TouchScreen > PaintExample.py > ...

```python

from kivy.app import App
from kivy.uix.widget import Widget
from kivy.graphics import Color, Ellipse, Line
from kivy.core.window import Window
import random

class DrawingPad(Widget):
    def color_for_touch(self, touch):
        if 'color' not in touch.ud:
            random.seed(touch.uid)
            touch.ud['color'] = (random.random(), random.random(), random.random())
        return touch.ud['color']

    def on_touch_down(self, touch):
        with self.canvas:
            Color(*self.color_for_touch(touch))
            d = 40  # circle diameter
            x, y = touch.pos
            touch.ud['circle'] = Ellipse(pos=(x - d/2, y - d/2), size=(d, d))
            touch.ud['line'] = Line(points=[x, y], width=2)

    def on_touch_move(self, touch):
        if 'line' in touch.ud:
            touch.ud['line'].points += touch.pos


class TouchDrawApp(App):
    def build(self):
        Window.clearcolor = (0.09, 0.09, 0.11, 1)  # dark background
        return DrawingPad()

if __name__ == "__main__":
    TouchDrawApp().run()

```

Thanks